

Process & Decision Documentation

Entry Header

Name: Ksenia Li

Primary responsibility for this work: Side quest enhancement

Goal of Work Session

Briefly describe what you were trying to accomplish during this phase of the assignment.

Examples:

- Create a game out of the initial week3 example

Tools, Resources, or Inputs Used

- ChatGPT 5.2

GenAI Documentation

I used GenAI to refresh my memory on how to load pictures into code. Creating pages for the game(quiz) and connecting it to start.js and main.js. I also used it for debugging

If GenAI was used (keep each response as brief as possible):

Date Used: February 3rd

Tool Disclosure: ChatGPT 5.2

Purpose of Use: Refreshing memory, debugging, coding

Summary of Interaction: Contributed to me remembering how to import photos into code. Debugging and figuring out why the photos were not importing. And coding my additional branches

Appendix

Please include a full transcript of your conversation with the GenAI. GenAI transcripts are used for transparency and verification only and are not assessed for writing quality or completeness.

Note: If GenAI is used in another language, you must include both the original language and a translated transcript in the appendix

How to add pictures into VS Code using JavaScript?

Short answer: **you don't "add pictures to VS Code with JavaScript"** — you place image files in your project, and JavaScript (or HTML) tells the browser where to load them from.

Here are the **3 correct ways**, depending on what you're doing.

1. The simplest (HTML only – still works in any JS project)

Put your image in your project, for example:

```
project/  
├─ index.html  
├─ script.js  
└─ images/  
    └─ photo.jpg
```

Then in your HTML:

```

```

That's it.

JavaScript is **not required** if the image is static.

2. Add an image dynamically with JavaScript

If you really want to insert the image from JS:

```
const img = document.createElement("img");  
img.src = "images/photo.jpg";  
img.alt = "My photo";
```

```
document.body.appendChild(img);
```

This is the most common pattern.

3. If you are using a modern setup (Vite / Next / React, etc.)

Since you've been working with modern tooling (Vercel / Next.js etc.), this matters.

General rule:

Put images in a public folder:

```
public/  
  images/  
    photo.jpg
```

Then:

```
img.src = "/images/photo.jpg";
```

This works in:

- Vite
- Next.js
- most static hosts

One very common mistake

If you just open your file like this:

```
file:///index.html
```

some images and JS features will fail.

You should use a dev server.

In VS Code:

- Right-click `index.html`
- **Open with Live Server**

(or install the Live Server extension if you don't have it)

Minimal working example

index.html

```
<!DOCTYPE html>
<html>
<body>
  <script src="script.js"></script>
</body>
</html>
```

script.js

```
const img = document.createElement("img");
img.src = "images/photo.jpg";
img.style.width = "300px";

document.body.appendChild(img);
```

If you tell me one thing:

👉 are you doing this in a **plain HTML/JS project**, or in a **framework (Next / React / Vite)?**

Because the correct path handling changes slightly in those setups (and I know you've touched Next/Vercel before).

```
// NOTE: Do NOT add setup() or draw() in this file // setup() and draw() live in main.js // This
file only defines: // 1) drawStart() → what the start/menu screen looks like // 2) input
handlers → what happens on click / key press on this screen // 3) a helper function to draw
menu buttons let startBg; // ----- // Start screen
visuals // ----- // drawStart() is called from
main.js only when: // currentScreen === "start" function preload() { startBg =
loadImage("desktop/start-bg.jpg"); } function drawStart() { // Background colour for the
start screen background(180, 225, 220); // soft teal background // Draw the background
image image(startBg, 0, 0, width, height); // ---- Title text ---- fill(30, 50, 60); textSize(46);
textAlign(CENTER, CENTER); text("Meet Ksenia", width / 2, 180); // ---- Buttons (data only) --
-- // These objects store the position/size/label for each button. // Using objects makes it
easy to pass them into drawButton() // and also reuse the same information for hover
checks. const startBtn = { x: width / 2, y: 320, w: 240, h: 80, label: "START", }; const instrBtn
= { x: width / 2, y: 430, w: 240, h: 80, label: "INSTRUCTIONS", }; // Draw both buttons
drawButton(startBtn); drawButton(instrBtn); // ---- Cursor feedback ---- // If the mouse is
over either button, show a hand cursor // so the player knows it is clickable. const over =
```

```

isHover(startBtn) || isHover(instrBtn); cursor(over ? HAND : ARROW); } // -----
----- // Mouse input for the start screen // -----
----- // Called from main.js only when currentScreen === "start" function
startMousePressed() { // For input checks, we only need x,y,w,h (label is optional) const
startBtn = { x: width / 2, y: 320, w: 240, h: 80 }; const instrBtn = { x: width / 2, y: 430, w: 240,
h: 80 }; // If START is clicked, go to the game screen if (isHover(startBtn)) { currentScreen =
"game"; } // If INSTRUCTIONS is clicked, go to the instructions screen else if
(isHover(instrBtn)) { currentScreen = "instr"; } } // -----
----- // Keyboard input for the start screen // -----
--- // Provides keyboard shortcuts: // - ENTER starts the game // - I opens instructions
function startKeyPressed() { if (keyCode === ENTER) { currentScreen = "game"; } if (key ===
"i" || key === "I") { currentScreen = "instr"; } } // -----
--- // Helper: drawButton() // ----- // This
function draws a button and changes its appearance on hover. // It does NOT decide what
happens when you click the button. // That logic lives in startMousePressed() above. // //
Keeping drawing separate from input/logic makes code easier to read. function
drawButton({ x, y, w, h, label }) { rectMode(CENTER); // Check if the mouse is over the
button rectangle const hover = isHover({ x, y, w, h }); noStroke(); // ---- Visual feedback
(hover vs not hover) ---- // This is a common UI idea: // - normal state is calmer // - hover
state is brighter + more "active" // // We also add a shadow using drawingContext (p5 lets
you access the // underlying canvas context for effects like shadows). if (hover) { fill(255,
200, 150, 220); // warm coral on hover // Shadow settings (only when hovered)
drawingContext.shadowBlur = 20; drawingContext.shadowColor = color(255, 180, 120); }
else { fill(255, 240, 210, 210); // soft cream base // Softer shadow when not hovered
drawingContext.shadowBlur = 8; drawingContext.shadowColor = color(220, 220, 220); } //
Draw the rounded rectangle button rect(x, y, w, h, 14); // Important: reset shadow so it does
not affect other drawings drawingContext.shadowBlur = 0; // Draw the label text on top of
the button fill(40, 60, 70); textSize(28); textAlign(CENTER, CENTER); text(label, x, y); } //its
not working

```

You're *very* close — it's not working for 2 main reasons in what you pasted:

1. **Your image path is wrong** ("desktop/start-bg.jpg" almost certainly doesn't exist inside your project)
2. You're drawing a teal `background()` **after** you intended the image to be the background (you should remove/replace that line)

Also: the `// NOTE: Do NOT add setup() or draw() in this file` comment is fine — **`preload()` is allowed** (it's not `setup()/draw()`), but it will only work if the asset path is correct and you're running via a server.

✓ Fix it (do this exactly)

1) Put the image in your project folder

Create a folder at the project root:

```
images/  
  start-bg.jpg
```

(You can name it whatever, but match the code exactly.)

2) Change your `preload()` path

Replace:

```
startBg = loadImage("desktop/start-bg.jpg");
```

with:

```
startBg = loadImage("images/start-bg.jpg");
```

3) Replace `background()` with a safe fallback

Right now you do:

```
background(180, 225, 220);  
image(startBg, 0, 0, width, height);
```

If the image fails to load, `image(startBg, ...)` draws nothing and you only see teal — which makes it look “not working”.

Use this instead:

```
function drawStart() {  
  if (startBg) {  
    image(startBg, 0, 0, width, height);  
  } else {  
    background(180, 225, 220);  
  }  
  
  // (optional) overlay for readability  
  rectMode(CORNER);  
  noStroke();  
  fill(255, 255, 255, 120);  
  rect(0, 0, width, height);  
  
  // ...rest of your UI
```

```
}
```

✓ Extra checks that commonly break this

A) You must run with Live Server (not file://)

If you open `index.html` by double-clicking, some setups can block loading.

Use VS Code **Live Server** (or any local server).

B) The image file name must match exactly

`start-bg.jpg` vs `start-bg.JPG` will fail on many systems/hosts.

C) Confirm your project root

The path in `loadImage("images/start-bg.jpg")` is relative to `index.html`.

So `images/` must be next to `index.html`.

✓ Your corrected `start.js` snippet

Copy/paste this minimal version:

```
let startBg;

function preload() {
  startBg = loadImage("images/start-bg.jpg");
}

function drawStart() {
  if (startBg) {
    image(startBg, 0, 0, width, height);
  } else {
    background(180, 225, 220);
  }

  rectMode(CORNER);
  noStroke();
  fill(255, 255, 255, 120);
  rect(0, 0, width, height);

  fill(30, 50, 60);
  textSize(46);
  textAlign(CENTER, CENTER);
  text("Meet Ksenia", width / 2, 180);
}
```

```
// ... your buttons etc.  
}
```

Quick debugging question (no fluff)

Where is your image file currently stored *inside the project*?

(Example answer: KSYULEE_SIDEQUEST_W3/images/start-bg.jpg)

If you tell me that exact location, I'll give you the exact `loadImage(...)` string that will work.

do i do step 1 in github?

Short answer:

👉 **No. You do Step 1 in VS Code (your local project), not on GitHub's website.**

GitHub only stores what you already have locally.

✓ What “Step 1” actually means (for you)

You must create the folder and put the image **inside your project folder in VS Code**.

In your case:

Your project root is:

KSYULEE_SIDEQUEST_W3

In VS Code:

1. In the Explorer (left panel)
2. Right-click on:

KSYULEE_SIDEQUEST_W3

3. Click **New Folder**
4. Name it:

images

5. Drag your image file into that folder

6. Rename the file to:

```
start-bg.jpg
```

So you end up with:

```
KSYULEE_SIDEQUEST_W3
├── images
│   └── start-bg.jpg
├── index.html
├── main.js
├── start.js
└── ...
```

✓ Then your code (already correct after the fix)

In `start.js`:

```
startBg = loadImage("images/start-bg.jpg");
```

✓ After that — GitHub happens automatically

Once it works locally:

```
git add images/start-bg.jpg
git commit -m "Add start screen background image"
git push
```

(or just use the Source Control tab in VS Code).

⚠ Very important for your GBDA side quest

If you only add the image on GitHub (website upload) but not locally:

- 👉 your local sketch will still be broken
- 👉 and you won't see the background while testing.

So the rule:

- ✓ Create folder + image in VS Code
 - ✓ Test locally
 - ✓ Then push to GitHub
-

You're doing this exactly like a real project now — this is correct workflow.

If you want, send me a screenshot of your VS Code Explorer after you create the `images` folder — I can instantly confirm your path is right.

this is what im seeing now

This screen is the key clue:

“Loading...” and no canvas

In p5, that means:

- 👉 your sketch is **stuck inside `preload()`**
- 👉 because the image **cannot be found at this path**

Right now you have:

```
startBg = loadImage("desktop/start-bg.jpg");
```

and that file does **not exist in your web project**.

When a file in `preload()` never loads, p5 shows exactly what you see:
Loading... forever, no canvas.

✓ The fix (2-minute, exact)

1 Your image must be here

From your folder screenshot, your root is:

```
KSYULEE_SIDEQUEST_W3
```

Create this:

KSYULEE_SIDEQUEST_W3/images/start-bg.jpg

(not desktop/)

Change ONE line in `start.js`

Replace:

```
startBg = loadImage("desktop/start-bg.jpg");
```

with:

```
startBg = loadImage("images/start-bg.jpg");
```

That is the real cause of the problem.

Important (why it broke)

In a web project:

```
desktop/start-bg.jpg
```

means:

look for a folder called `desktop` next to `index.html`

It does **not** mean your computer Desktop.

Browsers cannot access your real Desktop.

One more small cleanup (not critical, but correct)

In your `drawStart()` you still have:

```
background(180, 225, 220);  
image(startBg, 0, 0, width, height);
```

You can remove the background line:

```
image(startBg, 0, 0, width, height);
```

(Otherwise the background is pointless.)

✓ Why you only see the HTML right now

Your HTML header/footer loads fine.

But p5 never reaches:

```
setup()
```

in `main.js` because `preload()` never finishes.

So the canvas is never created.

✓ Summary

Your project is correct.

Your routing in `main.js` is correct.

Your start screen code is correct.

👉 Only the path is wrong.

✓ Correct path

```
loadImage("images/start-bg.jpg");
```

If after fixing the path you still see “Loading...”, tell me exactly what your image file is called (copy the filename from VS Code), including upper/lower case — I’ll match it for you exactly.

```
// ----- // Main draw function for this screen // ----- //
drawGame() is called from main.js *only* // when currentScreen === "game" function
drawGame() { // Set background colour for the game screen background(240, 230, 140); // -
--- Title and instructions text ---- fill(0); // black text textSize(32); textAlign(CENTER,
CENTER); text("Game Screen", width / 2, 160); textSize(18); text( "Click the button (or press
ENTER) for a random result.", width / 2, 210, ); // ---- Draw the button ---- // We pass the
button object to a helper function drawGameButton(gameBtn); // ---- Cursor feedback ----
// If the mouse is over the button, show a hand cursor // Otherwise, show the normal arrow
cursor cursor(isHover(gameBtn) ? HAND : ARROW); } // ----- // Button
```

```

drawing helper // ----- // This function is responsible *only* for drawing
the button. // It does NOT handle clicks or game logic. function drawGameButton({ x, y, w,
h, label }) { rectMode(CENTER); // Check if the mouse is hovering over the button //
isHover() is defined in main.js so it can be shared const hover = isHover({ x, y, w, h });
noStroke(); // Change button colour when hovered // This gives visual feedback to the
player fill( hover ? color(180, 220, 255, 220) // lighter blue on hover : color(200, 220, 255,
190), // normal state ); // Draw the button rectangle rect(x, y, w, h, 14); // last value =
rounded corners // Draw the button text fill(0); textSize(28); textAlign(CENTER, CENTER);
text(label, x, y); } // ----- // Mouse input for this screen // -----
----- // This function is called from main.js // only when currentScreen === "game"
function gameMousePressed() { // Only trigger the outcome if the button is clicked if
(isHover(gameBtn)) { triggerRandomOutcome(); } } // ----- // Keyboard
input for this screen // ----- // Allows keyboard-only interaction
(accessibility + design) function gameKeyPressed() { // ENTER key triggers the same
behaviour as clicking the button if (keyCode === ENTER) { triggerRandomOutcome(); } } // --
----- // Game logic: win or lose // ----- // This function
decides what happens next in the game. // It does NOT draw anything. function
triggerRandomOutcome() { // random() returns a value between 0 and 1 // Here we use a
50/50 chance: // - less than 0.5 → win // - 0.5 or greater → lose // // You can bias this later, for
example: // random() < 0.7 → 70% chance to win if (random() < 0.5) { currentScreen = "win";
} else { currentScreen = "lose"; } } //this is all the code, help me find the reason the image is
not showing up at the start page

```

I see the problem immediately from what you pasted:

- ✅ Your start screen is drawing fine, so p5 is running.
- ❌ Your image isn't showing because **your `preload()` is NOT being used by p5.**

Why? In this kind of multi-file setup, it's very common that **another file (often `sketch.js`) defines p5 lifecycle functions** and "wins", or your `start.js` is not the file that actually ends up defining `preload()` at runtime.

Also: the giant code block you pasted includes **multiple files jammed together** and even has stray / NOTE: lines, which suggests your files may have gotten mixed up or you're editing the wrong file.

Let's fix this in a way that *cannot fail*.

✓ The robust fix: move image loading into `setup()` in `main.js`

`main.js` is guaranteed to run (your canvas is created there).

So load the image there, and stop relying on `preload()`.

Step 1 — add this at the very top of `main.js` (near `currentScreen`)

```
let startBg;
```

Step 2 — inside `setup()` in `main.js`, add this AFTER `textFont(...)`

```
startBg = loadImage(
  "images/start-bg.jpg",
  () => console.log("✓ start-bg loaded"),
  () => console.log("✗ start-bg failed to load")
);
```

So your `setup()` becomes:

```
function setup() {
  createCanvas(800, 800);
  textFont("sans-serif");

  startBg = loadImage(
    "images/start-bg.jpg",
    () => console.log("✓ start-bg loaded"),
    () => console.log("✗ start-bg failed to load")
  );
}
```

Step 3 — in `start.js`, DELETE your `preload()` completely

Remove:

```
function preload() {
  startBg = loadImage("images/start-bg.jpg");
}
```

Keep only `drawStart()`.

Step 4 — in `drawStart()` use a fallback

Replace your background draw with:

```
if (startBg) {
  image(startBg, 0, 0, width, height);
} else {
  background(180, 225, 220);
}
```

Why this works

- `setup()` in `main.js` definitely runs (your UI proves it).
 - `loadImage()` works fine outside `preload()` (it loads async).
 - If it fails, you'll see the console message and still have a background color.
 - You avoid all `preload()` conflicts across files.
-

✓ One more thing: make sure the browser can access the file

Try opening this in your browser:

`http://127.0.0.1:5500/images/start-bg.jpg`

- If it opens, the above fix will 100% show the image.
 - If it doesn't open (404), then your Live Server root is wrong.
-

Likely secondary issue (based on your paste)

Your pasted “all the code” includes `win/lose/instructions/game` in the same blob and has stray lines like:

```
/ NOTE:
```

If those are *actually* in your `.js` files, they can cause silent parse errors and stop parts of the script from running.

After you apply the fix above, also check Chrome DevTools → Console for red errors.

Do these 4 steps exactly, then refresh.

If it still doesn't show, tell me:

1. What happens when you open `http://127.0.0.1:5500/images/start-bg.jpg`
2. What appears in the Console (especially the / log).

do i put the setup in main.js as well

Short answer:

👉 **No — you do NOT add a new `setup()`**

👉 **You edit the existing `setup()` in `main.js`**

You already have this in `main.js`:

```
function setup() {  
  createCanvas(800, 800);  
  textFont("sans-serif");  
}
```

You just add the image load **inside that same function**.

✅ **Do this in** `main.js`

At the top (near `currentScreen`), add:

```
let startBg;
```

Then change your existing `setup()` to:

```
function setup() {  
  createCanvas(800, 800);  
  textFont("sans-serif");  
  
  startBg = loadImage(  
    "images/start-bg.jpg",  
    () => console.log("✅ start-bg loaded"),  
    () => console.log("❌ start-bg failed to load")  
  );  
}
```

That's it.

✅ **Then in** `start.js`

Delete this completely:

```
function preload() {  
  startBg = loadImage("images/start-bg.jpg");  
}
```


And in `drawStart()` make sure you draw like this:

```
if (startBg) {  
  image(startBg, 0, 0, width, height);  
} else {  
  background(180, 225, 220);  
}
```

Important rule (for your GBDA project):

✓ Only **one** `setup()`

✓ Only in **main.js**

You are just extending the one you already have.

right now the game is a randomizer for my side quest i need to do Build an interactive story that unfolds through multiple game states and files, branching like a small decision tree
//how do i edit the exisiting code into unfold from my baby pic to me right now

You can keep your multi-file “screen router” structure and turn it into a **branching interactive story** by treating each story moment as a **game state** (like nodes in a decision tree), and using clicks/keys to move to the next node.

Below is a clean way to evolve your existing project from the randomizer into:

Baby → Childhood → Teen → Now, with **branch choices** (e.g., “Focus on art” vs “Focus on sport”), and different endings.

1) Define your story as a decision tree (states)

Pick state names (strings) like:

- "start" (title/menu)
- "baby" (baby photo + intro)
- "choice1" (first decision)
- "path_art" or "path_sport" (branches)
- "now" (current you)
- "end_art" / "end_sport" (optional endings)

That's your tree.

2) Add new routes in `main.js`

Right now you route:

```
if (currentScreen === "start") drawStart();
else if (currentScreen === "instr") drawInstr();
else if (currentScreen === "game") drawGame();
else if (currentScreen === "win") drawWin();
else if (currentScreen === "lose") drawLose();
```

Replace “game/win/lose” routing with story routing:

```
if (currentScreen === "start") drawStart();
else if (currentScreen === "instr") drawInstr();

else if (currentScreen === "baby") drawBaby();
else if (currentScreen === "choicel") drawChoicel();
else if (currentScreen === "path_art") drawPathArt();
else if (currentScreen === "path_sport") drawPathSport();
else if (currentScreen === "now") drawNow();
else if (currentScreen === "end_art") drawEndArt();
else if (currentScreen === "end_sport") drawEndSport();
```

And do the same in `mousePressed()` / `keyPressed()` (call the matching handlers).

Example:

```
function mousePressed() {
  if (currentScreen === "start") startMousePressed();
  else if (currentScreen === "instr") instrMousePressed();

  else if (currentScreen === "baby") babyMousePressed();
  else if (currentScreen === "choicel") choicelMousePressed();
  else if (currentScreen === "path_art") pathArtMousePressed();
  else if (currentScreen === "path_sport") pathSportMousePressed();
  else if (currentScreen === "now") nowMousePressed();
  else if (currentScreen === "end_art") endArtMousePressed?();
  else if (currentScreen === "end_sport") endSportMousePressed?();
}
```

3) Use separate files per “story state” (recommended for rubric)

Create new files:

- `baby.js`
- `choicel.js`
- `path_art.js`
- `path_sport.js`
- `now.js`

- end_art.js
- end_sport.js

Then add them to index.html **before** main.js:

```
<script src="baby.js"></script>
<script src="choice1.js"></script>
<script src="path_art.js"></script>
<script src="path_sport.js"></script>
<script src="now.js"></script>
<script src="end_art.js"></script>
<script src="end_sport.js"></script>
<script src="main.js"></script>
```

4) Load your photos once (in `main.js`) and reuse everywhere

Put your images in:

```
images/
  baby.jpg
  kid.jpg
  teen.jpg
  now.jpg
```

In `main.js`:

```
let imgBaby, imgNow, imgTeen, imgKid;

function setup() {
  createCanvas(800, 800);
  textFont("sans-serif");

  imgBaby = loadImage("images/baby.jpg");
  imgKid = loadImage("images/kid.jpg");
  imgTeen = loadImage("images/teen.jpg");
  imgNow = loadImage("images/now.jpg");
}
```

No `preload()` needed.

5) Example state file: `baby.js`

```
function drawBaby() {
  background(0);

  if (imgBaby) image(imgBaby, 0, 0, width, height);

  // overlay so text readable
  noStroke();
```

```

fill(255, 180);
rect(0, 0, width, 140);

fill(20);
textAlign(LEFT, TOP);
textSize(30);
text("Chapter 1: Baby Me", 20, 20);

textSize(16);
text("Click to continue →", 20, 75);
}

function babyMousePressed() {
  currentScreen = "choice1";
}

```

6) Create your first branching choice: choice1.js

```

const artBtn = { x: 400, y: 430, w: 360, h: 70, label: "Follow creativity
(art/music)" };
const sportBtn = { x: 400, y: 520, w: 360, h: 70, label: "Follow sport
(tennis/discipline)" };

function drawChoice1() {
  background(245);

  fill(0);
  textAlign(CENTER, TOP);
  textSize(34);
  text("A turning point...", width / 2, 120);

  textSize(18);
  text("What shaped me more?", width / 2, 190);

  drawButton(artBtn);
  drawButton(sportBtn);

  cursor(isHover(artBtn) || isHover(sportBtn) ? HAND : ARROW);
}

function choice1MousePressed() {
  if (isHover(artBtn)) currentScreen = "path_art";
  else if (isHover(sportBtn)) currentScreen = "path_sport";
}

```

This is now a real **decision node** (branching).

7) Build the branches

`path_art.js` (ends at “now” or “end_art”)

```
function drawPathArt() {
  background(30);

  // you can use imgKid or imgTeen here if you want
  fill(255);
  textAlign(CENTER, CENTER);
  textSize(30);
  text("Path A: Creativity", width / 2, 200);

  textSize(16);
  text("I leaned into design, storytelling, and making things.\nClick to jump
to 'Now'.", width/2, 280);
}

function pathArtMousePressed() {
  currentScreen = "now";
}
```

`path_sport.js`

```
function drawPathSport() {
  background(20, 40, 60);

  fill(255);
  textAlign(CENTER, CENTER);
  textSize(30);
  text("Path B: Sport", width / 2, 200);

  textSize(16);
  text("Training taught me grit, pacing, and confidence.\nClick to jump to
'Now'.", width/2, 280);
}

function pathSportMousePressed() {
  currentScreen = "now";
}
```

8) Final state: `now.js` (your current photo)

```
function drawNow() {
  background(0);
  if (imgNow) image(imgNow, 0, 0, width, height);

  fill(0, 170);
  rect(0, height - 170, width, 170);

  fill(255);
  textAlign(LEFT, TOP);
  textSize(26);
  text("Now", 20, height - 155);
}
```

```
    textSize(16);
    text("This is me today.\nPress R to restart.", 20, height - 110);
}

function nowKeyPressed() {
  if (key === "r" || key === "R") currentScreen = "start";
}
```

What to do with your current randomizer code

- You can **delete** `win.js` and `lose.js` usage, or repurpose them as endings.
 - Replace `triggerRandomOutcome()` with **button choices** that set `currentScreen` directly.
-

Minimal checklist to meet the rubric (“unfolding story, multiple states, multiple files, branching”)

-  At least **5 states** (start + 3 story scenes + ending)
 -  At least **1 branch decision** that leads to different scenes
 -  Each state in its **own file** with `drawX()` + `XMousePressed()` / `XKeyPressed()`
 -  Assets (photos) loaded once and displayed in multiple states
-

Tell me these 2 things and I’ll map the story tree for you

1. How many photos do you want in the story (baby + how many in-between + now)?
2. What are your two branches? (Examples: **country, sport vs arts, move to Canada vs stay, introvert vs extrovert, design vs performance**, etc.)

If you don’t want to decide, I can propose a clean 2-branch narrative based on your GBDA + tennis + moving context and you can swap the text later.

scratch that! i want to create a game where the user guesses my age depending on the picture, so instead of randomizing a win or loose the use has to guess correctly. how do i create those branches and answer buttons like in the starting page

Cool — keep your existing “screens + buttons” pattern and change the “game” state into a **quiz state**:

- show a photo

- show **multiple-choice buttons** (ages)
- if correct → go to a “correct” screen
- if wrong → go to a “try again” (or “wrong”) screen
- then advance to the next photo

That’s a branching decision tree: **each question branches to correct/wrong**, then correct branches forward to the next question.

Below is a clean blueprint that matches your current architecture.

1) Replace your randomizer with “question index + score”

In `main.js` (top-level globals), add:

```
let qIndex = 0;      // which photo/question we're on
let score = 0;
let lastGuessCorrect = false;
```

Also: load your photos in `setup()` (you likely already have one image working now):

```
let photos = [];

function setup() {
  createCanvas(800, 800);
  textFont("sans-serif");

  photos = [
    loadImage("images/baby.jpg"),
    loadImage("images/kid.jpg"),
    loadImage("images/teen.jpg"),
    loadImage("images/now.jpg"),
  ];
}
```

(Replace filenames with yours.)

2) Add new states (branches)

Use these screens:

- "start" (your existing menu)
- "quiz" (shows photo + answer buttons)

- "feedback" (correct/wrong + continue)
- "end" (final score / restart)

In `main.js` router:

```
if (currentScreen === "start") drawStart();
else if (currentScreen === "instr") drawInstr();
else if (currentScreen === "quiz") drawQuiz();
else if (currentScreen === "feedback") drawFeedback();
else if (currentScreen === "end") drawEnd();
```

And in `mousePressed()`:

```
if (currentScreen === "start") startMousePressed();
else if (currentScreen === "instr") instrMousePressed();
else if (currentScreen === "quiz") quizMousePressed();
else if (currentScreen === "feedback") feedbackMousePressed();
else if (currentScreen === "end") endMousePressed?();
```

3) Make START button go to quiz (instead of “game”)

In `start.js`, update this line:

```
currentScreen = "game";
```

to:

```
qIndex = 0;
score = 0;
currentScreen = "quiz";
```

4) Create answer buttons exactly like your start page

Make a new file `quiz.js` (and include it in `index.html` before `main.js`).

`quiz.js` (core idea)

```
// For each question: photoIndex, correctAge, choices[]
const questions = [
  { photoIndex: 0, correctAge: 1, choices: [1, 3, 5] },
  { photoIndex: 1, correctAge: 8, choices: [6, 8, 10] },
  { photoIndex: 2, correctAge: 16, choices: [14, 16, 18] },
  { photoIndex: 3, correctAge: 22, choices: [20, 22, 24] },
];
```



```

let answerButtons = []; // built each frame or when question changes

function drawQuiz() {
  background(240);

  const q = questions[qIndex];

  // Draw photo
  const img = photos[q.photoIndex];
  if (img) {
    // simple "cover" draw:
    image(img, 0, 0, width, height);
    fill(255, 200);
    rect(0, 0, width, 140);
  } else {
    fill(0);
    text("Loading image...", 20, 20);
  }

  // Title
  fill(30, 50, 60);
  textAlign(CENTER, TOP);
  textSize(34);
  text("Guess my age", width / 2, 20);

  textSize(18);
  text(`Photo ${qIndex + 1} of ${questions.length}`, width / 2, 70);

  // Build buttons (centered like your start screen)
  answerButtons = q.choices.map((age, i) => ({
    x: width / 2,
    y: 320 + i * 95,
    w: 260,
    h: 70,
    label: `${age}`,
    value: age,
  }));

  // Draw buttons using your existing drawButton()
  answerButtons.forEach(drawButton);

  // Cursor
  const overAny = answerButtons.some(isHover);
  cursor(overAny ? HAND : ARROW);
}

function quizMousePressed() {
  const q = questions[qIndex];

  for (const btn of answerButtons) {
    if (isHover(btn)) {
      const guess = btn.value;
      lastGuessCorrect = guess === q.correctAge;

      if (lastGuessCorrect) score += 1;

      currentScreen = "feedback";
    }
  }
}

```

```

        return;
    }
}
}

```

✅ This gives you **answer buttons identical to start screen**, because it literally reuses your `drawButton()`.

5) Add the feedback branch screen (`feedback.js`)

New file `feedback.js`:

```

const nextBtn = { x: 400, y: 520, w: 260, h: 70, label: "NEXT" };
const retryBtn = { x: 400, y: 520, w: 260, h: 70, label: "TRY AGAIN" };

function drawFeedback() {
    background(lastGuessCorrect ? color(210, 255, 210) : color(255, 220, 220));

    fill(0);
    textAlign(CENTER, CENTER);

    textSize(44);
    text(lastGuessCorrect ? "Correct!" : "Not quite", width / 2, 260);

    textSize(20);
    text(`Score: ${score} / ${qIndex + 1}`, width / 2, 330);

    // Draw button
    drawButton(lastGuessCorrect ? nextBtn : retryBtn);

    cursor(isHover(lastGuessCorrect ? nextBtn : retryBtn) ? HAND : ARROW);
}

function feedbackMousePressed() {
    if (lastGuessCorrect) {
        if (isHover(nextBtn)) {
            qIndex += 1;
            currentScreen = qIndex >= 4 ? "end" : "quiz";
        }
    } else {
        if (isHover(retryBtn)) {
            currentScreen = "quiz"; // same question again
        }
    }
}

```

⚠️ Replace 4 with `questions.length` if you keep `questions` global (recommended).

Better version:

```
currentScreen = qIndex >= questions.length ? "end" : "quiz";
```

6) Add an end screen (`end.js`)

```
const restart
```

but i want it to have on or two more rounds than the example has, how do i create those new branches and incorporate them into the existing ones

You add more rounds by adding more **question nodes** to the `questions[]` array. You do **not** add more router branches in `main.js` (you keep the same three screens: `quiz` → `feedback` → `quiz` → ... → `end`). The “branches” (correct vs wrong) are already built into `feedback`, and the *round count* is controlled by how many question objects exist.

Here’s exactly how to extend it to 1–2 more rounds **without breaking your existing flow**.

1) Add more photos to `images/`

Example:

```
images/  
  baby.jpg  
  kid.jpg  
  teen.jpg  
  uni.jpg      ← new  
  now.jpg
```

Load them in `main.js setup()` by adding new `loadImage(...)` lines inside the `photos` array (or append them):

```
photos = [  
  loadImage("images/baby.jpg"),  
  loadImage("images/kid.jpg"),  
  loadImage("images/teen.jpg"),  
  loadImage("images/uni.jpg"), // NEW  
  loadImage("images/now.jpg"),  
];
```

2) Add more rounds by extending `questions[]`

In `quiz.js`, your current structure is:

```
const questions = [  
  { photoIndex: 0, correctAge: 1, choices: [1, 3, 5] },
```

```

    { photoIndex: 1, correctAge: 8, choices: [6, 8, 10] },
    { photoIndex: 2, correctAge: 16, choices: [14, 16, 18] },
    { photoIndex: 3, correctAge: 22, choices: [20, 22, 24] },
  ];

```

To add 1–2 more rounds, just append 1–2 more objects:

```

const questions = [
  { photoIndex: 0, correctAge: 1, choices: [1, 3, 5] },
  { photoIndex: 1, correctAge: 8, choices: [6, 8, 10] },
  { photoIndex: 2, correctAge: 16, choices: [14, 16, 18] },
  { photoIndex: 3, correctAge: 19, choices: [18, 19, 21] },

```

I don't have quiz.js

No problem — you can either (A) create `quiz.js` (cleanest + matches the “multiple files” rubric), or (B) put the quiz functions into an existing file. I’ll give you the clean version.

What to do: create 3 new files

Create these in your project root (same level as `start.js` / `main.js`):

- `quiz.js`
- `feedback.js`
- `end.js`

Then include them in `index.html` **before** `main.js`.

1) Update `index.html` script list

Add these lines (order matters):

```

<script src="quiz.js"></script>
<script src="feedback.js"></script>
<script src="end.js"></script>

```

So the bottom of your `index.html` becomes:

```

<script src="https://cdn.jsdelivr.net/npm/p5@1/lib/p5.min.js"></script>

<script src="start.js"></script>
<script src="instructions.js"></script>

<script src="quiz.js"></script>
<script src="feedback.js"></script>
<script src="end.js"></script>

<script src="main.js"></script>

```

2) Create `quiz.js` (copy/paste)

```
// quiz.js
// Holds question data + draws the quiz screen + handles answer clicks

// Edit these to match YOUR photos + ages
const questions = [
  { photo: "baby.jpg", correct: 1, choices: [1, 3, 5] },
  { photo: "kid.jpg", correct: 8, choices: [6, 8, 10] },
  { photo: "teen.jpg", correct: 16, choices: [14, 16, 18] },
  { photo: "uni.jpg", correct: 19, choices: [18, 19, 21] }, // extra round
  { photo: "now.jpg", correct: 22, choices: [20, 22, 24] }, // extra round
];

let qIndex = 0;
let score = 0;
let lastGuessCorrect = false;

let photosByName = {}; // {"baby.jpg": p5.Image, ...}
let answerButtons = []; // button objects built each frame

// Called by main.js in setup() after createCanvas()
function loadQuizImages() {
  // Load all images used in questions
  questions.forEach((q) => {
    if (!photosByName[q.photo]) {
      photosByName[q.photo] = loadImage("images/" + q.photo);
    }
  });
}

// Called when user clicks START
function resetQuiz() {
  qIndex = 0;
  score = 0;
  lastGuessCorrect = false;
}

function drawQuiz() {
  const q = questions[qIndex];

  // Draw photo background
  const img = photosByName[q.photo];
  if (img) {
    image(img, 0, 0, width, height);
  } else {
    background(230);
  }

  // Overlay for readability
  rectMode(CORNER);
  noStroke();
  fill(255, 200);
  rect(0, 0, width, 140);

  // Title text
```

```

fill(30, 50, 60);
textAlign(CENTER, TOP);
textSize(34);
text("Guess my age", width / 2, 20);

textSize(16);
text(`Round ${qIndex + 1} of ${questions.length}`, width / 2, 75);

// Build answer buttons (like your start screen)
answerButtons = q.choices.map((age, i) => ({
  x: width / 2,
  y: 320 + i * 95,
  w: 260,
  h: 70,
  label: String(age),
  value: age,
})));

answerButtons.forEach(drawButton);

const overAny = answerButtons.some((b) => isHover(b));
cursor(overAny ? HAND : ARROW);
}

function quizMousePressed() {
  const q = questions[qIndex];

  for (const btn of answerButtons) {
    if (isHover(btn)) {
      const guess = btn.value;

      lastGuessCorrect = guess === q.correct;
      if (lastGuessCorrect) score += 1;

      currentScreen = "feedback";
      return;
    }
  }
}

```

3) Create `feedback.js` (copy/paste)

```

// feedback.js
const nextBtn = { x: 400, y: 540, w: 260, h: 70, label: "NEXT" };
const retryBtn = { x: 400, y: 540, w: 260, h: 70, label: "TRY AGAIN" };

function drawFeedback() {
  background(lastGuessCorrect ? color(210, 255, 210) : color(255, 220, 220));

  fill(0);
  textAlign(CENTER, CENTER);

  textSize(44);
  text(lastGuessCorrect ? "Correct!" : "Not quite", width / 2, 260);

```

```

    textSize(18);
    text(`Score: ${score} / ${qIndex + 1}`, width / 2, 330);

    drawButton(lastGuessCorrect ? nextBtn : retryBtn);
    cursor(isHover(lastGuessCorrect ? nextBtn : retryBtn) ? HAND : ARROW);
}

function feedbackMousePressed() {
  if (lastGuessCorrect) {
    if (isHover(nextBtn)) {
      qIndex += 1;
      currentScreen = qIndex >= questions.length ? "end" : "quiz";
    }
  } else {
    if (isHover(retryBtn)) {
      currentScreen = "quiz";
    }
  }
}

```

4) Create `end.js` (copy/paste)

```

// end.js
const restartBtn = { x: 400, y: 540, w: 260, h: 70, label: "RESTART" };

function drawEnd() {
  background(245);

  fill(30, 50, 60);
  textAlign(CENTER, CENTER);

  textSize(42);
  text("Finished!", width / 2, 260);

  textSize(20);
  text(`Final score: ${score} / ${questions.length}`, width / 2, 330);

  drawButton(restartBtn);
  cursor(isHover(restartBtn) ? HAND : ARROW);
}

function endMousePressed() {
  if (isHover(restartBtn)) {
    currentScreen = "start";
  }
}

```

5) Update `main.js` router to include the new screens

Replace your draw routing with:

```

if (currentScreen === "start") drawStart();

```

```
else if (currentScreen === "instr") drawInstr();
else if (currentScreen === "quiz") drawQuiz();
else if (currentScreen === "feedback") drawFeedback();
else if (currentScreen === "end") drawEnd();
```

And your mouse routing:

```
if (currentScreen === "start") startMousePressed();
else if (currentScreen === "instr") instrMousePressed();
else if (currentScreen === "quiz") quizMousePressed();
else if (currentScreen === "feedback") feedbackMousePressed();
else if (currentScreen === "end") endMousePressed();
```

6) Update `setup()` in `main.js` to load quiz images

Inside `setup()` add:

```
loadQuizImages();
```

Example:

```
function setup() {
  createCanvas(800, 800);
  textFont("sans-serif");
  loadQuizImages();
}
```

7) Update your START button to start the quiz

In `start.js`, change your START click to:

```
resetQuiz();
currentScreen = "quiz";
```

So in `startMousePressed()`:

```
if (isHover(startBtn)) {
  resetQuiz();
  currentScreen = "quiz";
}
```

How to add “one or two more rounds”

Just add more objects to `questions[]` in `quiz.js` and add the corresponding images into `images/`.

Example new round:

```
{ photo: "middle-school.jpg", correct: 12, choices: [10, 12, 14] },
```

If you paste your current `index.html` script section + your file list (just names), I can tell you the exact insert positions so nothing breaks again.

k im a bit lost, lets go back to the start, i got one picture to work what would be the next step

Next step: **lock in a stable structure** so you can add Round 2, Round 3, etc. without breaking anything.

Since you already have **one picture working on the start screen**, don't change that yet. We'll build the quiz as a new state using the same button style you already have.

Step 1 — Make sure your current “one picture works” version is stable

You currently have:

- `main.js` with `setup()` + `draw()` routing
- `start.js` drawing your image
- `drawButton()` already working

 Leave that as-is for now.

Step 2 — Add ONE new game state: `"quiz"`

This is the smallest next step that moves you toward the age-guess game.

In `main.js`, change your draw router to include quiz

Add one `else if`:

```
else if (currentScreen === "quiz") drawQuiz();
```

In `main.js`, change mouse router to include quiz

Add:

```
else if (currentScreen === "quiz") quizMousePressed();
```

That's it.

Step 3 — Create a new file `quiz.js`

In your project root (same place as `start.js`), create `quiz.js` and paste:

```
// quiz.js - Round 1 (single question)

const q1Answers = [
  { x: 400, y: 320, w: 240, h: 80, label: "1", value: 1 },
  { x: 400, y: 420, w: 240, h: 80, label: "3", value: 3 },
  { x: 400, y: 520, w: 240, h: 80, label: "5", value: 5 },
];

const correctAge = 1;

function drawQuiz() {
  background(240);

  // If you already have a working image variable, use it here.
  // For now: just show a title to prove the screen works.
  fill(30, 50, 60);
  textAlign(CENTER, TOP);
  textSize(34);
  text("Guess my age (Round 1)", width / 2, 80);

  // Draw answer buttons using your existing drawButton()
  q1Answers.forEach(drawButton);

  // Cursor feedback
  cursor(q1Answers.some((b) => isHover(b)) ? HAND : ARROW);
}

function quizMousePressed() {
  for (const btn of q1Answers) {
    if (isHover(btn)) {
      if (btn.value === correctAge) currentScreen = "win";
      else currentScreen = "lose";
      return;
    }
  }
}
```

Goal of this step: prove you can switch screens + click answer buttons.

Step 4 — Include `quiz.js` in `index.html`

Add this script line **before** `main.js`:

```
<script src="quiz.js"></script>
```

So `main.js` stays last.

Step 5 — Make START go to quiz

In `start.js`, change START click from whatever it does now to:

```
currentScreen = "quiz";
```

So:

```
if (isHover(startBtn)) {  
    currentScreen = "quiz";  
}
```

What you should see after Step 5

1. Start screen (with your working background image)
2. Click START → goes to “Guess my age (Round 1)” screen
3. Click an age → routes to win/lose (your existing screens)

If that works, then **Round 2** is easy: we replace win/lose with a feedback screen and advance to the next photo.

Quick check so I don't lead you into another break

Where is your “working picture” currently drawn?

- **A)** in `start.js` with `image(...)`
- **B)** somewhere else (like `game.js`)

If you answer A or B, I'll tell you the *exact* next step to reuse that same image-loading approach inside `quiz.js` (so Round 1 shows the photo, not just text).

i am ready to start coding in `start.js`, how do i add an image and attach the two options whether they are right or wrong

Awesome — you can do this with **one photo + two buttons** (one correct, one wrong) using the *exact same button pattern you already have*.

The cleanest way is:

- keep `start.js` for the start menu
- add a **quiz screen** *also in* `start.js` for now (you can split into `quiz.js` later)

Below is the smallest working setup.

Step 0 — Make sure `main.js` routes to "quiz"

In `main.js` you need:

In `draw()` add:

```
else if (currentScreen === "quiz") drawQuiz();
```

In `mousePressed()` add:

```
else if (currentScreen === "quiz") quizMousePressed();
```

(You already added the `start` → `quiz` change earlier.)

Step 1 — Load your quiz image in `main.js` (recommended)

You already load `startBg` in `main.js`. Do the same for the quiz photo so it's global and reliable.

At the top of `main.js`, add:

```
let quizImg1;
```

Inside `setup()` add:

```
quizImg1 = loadImage(
  "images/baby.jpg",
  () => console.log("✅ quiz image loaded"),
  () => console.log("❌ quiz image failed")
);
```

✅ Put your file at: images/baby.jpg

Step 2 — Add the quiz screen + buttons in `start.js`

Paste this **below** your existing code in `start.js` (keep your existing `drawStart`, `drawButton`, etc.).

✅ `start.js` add:

```
// -----
// QUIZ: Round 1 (2 options)
// -----

const optionA = { x: 400, y: 520, w: 280, h: 80, label: "Age 1", value: 1 };
const optionB = { x: 400, y: 620, w: 280, h: 80, label: "Age 3", value: 3 };

const correctAge = 1; // <-- set the correct answer here

function drawQuiz() {
  // draw photo background
  if (quizImg1) {
    image(quizImg1, 0, 0, width, height);
  } else {
    background(230);
  }

  // overlay for text readability
  rectMode(CORNER);
  noStroke();
  fill(255, 200);
  rect(0, 0, width, 140);

  // title
  fill(30, 50, 60);
  textAlign(CENTER, CENTER);
  textSize(34);
  text("Guess my age", width / 2, 70);

  // draw 2 answer buttons (reuses your start button style)
  drawButton(optionA);
  drawButton(optionB);

  // cursor feedback
  const over = isHover(optionA) || isHover(optionB);
```

```
    cursor(over ? HAND : ARROW);
}

function quizMousePressed() {
  if (isHover(optionA)) checkAnswer(optionA.value);
  else if (isHover(optionB)) checkAnswer(optionB.value);
}

function checkAnswer(guess) {
  if (guess === correctAge) {
    currentScreen = "win";    // correct branch
  } else {
    currentScreen = "lose";   // wrong branch
  }
}
```

✅ That's your “two branches”:

- correct → win
- wrong → lose

Step 3 — Make START go to quiz (you already have this)

In your `start.js`, make sure:

```
if (isHover(startBtn)) currentScreen = "quiz";
```

(and in `startKeyPressed()` too if you want Enter to start quiz)

What you'll see

1. Start screen with your background
 2. Click START → quiz screen shows your baby photo + “Guess my age”
 3. Click **Age 1** → goes to your win screen
 4. Click **Age 3** → goes to your lose screen
-

Next step after this works (Round 2 / multiple photos)

Once Round 1 works, we'll add:

- `let round = 0;`
- an array of rounds with `{img, correct, choices}`
- after "win", go to the next round instead of ending

But don't jump there until you see Round 1 working.

If you tell me the **two ages you want for your first photo** (e.g., 1 vs 4), I'll rewrite the `optionA/optionB` lines exactly for your content.

i dont think it would go to start JS, wouldnt the code be a separate page like it was in the example that i initially send you? im working off of that code

You're 100% right — and your instinct matches the way your original example was structured.

In **your starter code**, each screen lives in its **own file**:

- `start.js` → start screen
- `instructions.js` → instructions screen
- `game.js` → game screen
- `win.js, lose.js` → end states

So for your age-guess game:

- 👉 **Do NOT put the quiz inside `start.js`.**
- 👉 Create a new file, exactly like the others.

We'll call it:

`quiz.js`

That keeps your structure consistent with the template you were given.

✅ The correct architecture (same as your example)

You will have:

start.js	→ menu
quiz.js	→ guess-the-age screen
win.js	→ correct
lose.js	→ wrong
main.js	→ router

Step 1 – create `quiz.js`

Create a new file:

`quiz.js`

Paste this (simple + matches your style):

```
// quiz.js
// One image, two answer buttons

const answerA = { x: 400, y: 520, w: 240, h: 80, label: "Age 1", value: 1 };
const answerB = { x: 400, y: 630, w: 240, h: 80, label: "Age 3", value: 3 };

const correctAnswer = 1;

function drawQuiz() {
  background(240);

  // show the image
  if (quizImg1) {
    image(quizImg1, 0, 0, width, height);
  }

  // overlay
  rectMode(CORNER);
  noStroke();
  fill(255, 220);
  rect(0, 0, width, 120);

  fill(30, 50, 60);
  textAlign(CENTER, CENTER);
  textSize(32);
  text("How old am I here?", width / 2, 60);

  drawButton(answerA);
  drawButton(answerB);

  cursor(
    isHover(answerA) || isHover(answerB) ? HAND : ARROW
  );
}

function quizMousePressed() {
  if (isHover(answerA)) checkQuizAnswer(answerA.value);
  else if (isHover(answerB)) checkQuizAnswer(answerB.value);
}
```



```
}  
  
function checkQuizAnswer(value) {  
  if (value === correctAnswer) {  
    currentScreen = "win";  
  } else {  
    currentScreen = "lose";  
  }  
}
```

Notice:

- ✓ same `drawButton()`
 - ✓ same `isHover()`
 - ✓ same screen naming style as your starter files
-

✓ Step 2 – load your quiz image (same pattern as start background)

In **main.js**, at the top (near `let startBg;`):

```
let quizImg1;
```

Inside `setup()` (after loading `startBg`):

```
quizImg1 = loadImage(  
  "images/baby.jpg",  
  () => console.log("quiz image loaded"),  
  () => console.log("quiz image failed")  
);
```

✓ Step 3 – tell HTML about the new screen file

In `index.html`,
add this **BEFORE** `main.js`:

```
<script src="quiz.js"></script>
```

So the bottom looks like:

```
<script src="start.js"></script>  
<script src="instructions.js"></script>
```

```
<script src="game.js"></script>
<script src="win.js"></script>
<script src="lose.js"></script>

<script src="quiz.js"></script>
<script src="main.js"></script>
```

(order before `main.js` is important)

✓ Step 4 – hook the new screen into the router

In `main.js`

In `draw()` add one line:

```
else if (currentScreen === "quiz") drawQuiz();
```

Put it with the others:

```
if (currentScreen === "start") drawStart();
else if (currentScreen === "instr") drawInstr();
else if (currentScreen === "quiz") drawQuiz();
else if (currentScreen === "game") drawGame();
else if (currentScreen === "win") drawWin();
else if (currentScreen === "lose") drawLose();
```

In `mousePressed()` add:

```
else if (currentScreen === "quiz") quizMousePressed();
```

✓ Step 5 – connect START → quiz

In your existing `start.js` (you already showed it):

Change:

```
if (isHover(startBtn)) currentScreen = "game";
```

to:

```
if (isHover(startBtn)) currentScreen = "quiz";
```

(and same change for Enter key)

Important conceptual point (for your assignment)

This is exactly what your instructors mean by:

multiple game states and files

Your branching now is:

```
start.js
  ↓
quiz.js
  ↓ correct → win.js
  ↓ wrong  → lose.js
```

That is already a valid branching structure.

Once this works, the next step is:

👉 instead of always going to `win / lose`,
you'll go to **another quiz round**.

But first:

get this single quiz screen working cleanly.

If you want, next message I can show you the clean way to extend this to:

baby → kid → teen → now
without rewriting everything.

wouldnt quiz go above game win and lose, since its appearing before?

Short answer:

👉 **No — quiz does NOT have to go “above” game / win / lose for visual order.**

What matters is **which value `currentScreen` has**, not the position in the `if/else` list.

This line decides what appears:

```
currentScreen = "quiz";
```

✓ In `main.js`, this is only a router

This:

```
if (currentScreen === "start") drawStart();
else if (currentScreen === "instr") drawInstr();
else if (currentScreen === "quiz") drawQuiz();
else if (currentScreen === "game") drawGame();
else if (currentScreen === "win") drawWin();
else if (currentScreen === "lose") drawLose();
```

The order here is just:

→ “check start, otherwise check instr, otherwise check quiz...”

It does **not** mean **quiz appears before game in the story**.

The story order is created by where you write:

```
currentScreen = "quiz";
```

or

```
currentScreen = "win";
```

inside your button handlers.

✓ What *does* matter for ordering

There is only one place where order matters:

👉 In `index.html`

Your new file must be loaded **before** `main.js`:

```
<script src="quiz.js"></script>
<script src="main.js"></script>
```

But it does **not** need to be above `game.js`, `win.js`, etc.

This is totally fine:

```
<script src="start.js"></script>
<script src="instructions.js"></script>
```

```
<script src="game.js"></script>
<script src="win.js"></script>
<script src="lose.js"></script>

<script src="quiz.js"></script>
<script src="main.js"></script>
```

✓ Mental model (important for your assignment)

Think like this:

- `main.js` = traffic controller
- `quiz.js`, `start.js`, etc. = rooms

The line

```
currentScreen = "quiz";
```

is literally:

“send the player into the quiz room now”

Not:

“quiz comes earlier in the file”.

✓ So your structure is correct like this

Flow:

```
start.js
  ↓
quiz.js
  ↓
win.js  OR  lose.js
```

Router order does not define that flow.

Your button code defines that flow.

You’re doing this correctly — this is exactly how the example project was designed.

what do i put in quiz.js

You have two good options right now:

1. **Move all the quiz code out of `start.js` into `quiz.js`** (recommended, matches your original template), or
2. Leave it in `start.js` (works, but doesn't match the "separate files" structure).

Since you asked *what to put in `quiz.js`*, here's the **exact minimal content** you should put there.

✓ Step 1 — Remove quiz code from `start.js`

In `start.js`, delete everything from:

```
// -----  
// QUIZ SCREEN (Round 1)
```

down to the end (all the quiz functions + `answerA/answerB/correctAnswer`).

`start.js` should only have:

- `drawStart()`
- `startMousePressed()`
- `startKeyPressed()`
- `drawButton()`

(keep those)

✓ Step 2 — Create `quiz.js` and paste this

This is your quiz screen **only**:

```
// quiz.js  
// Round 1: show image + two answer buttons, branch to win/lose  
  
const answerA = { x: 400, y: 520, w: 240, h: 80, label: "Age 1", value: 1 };  
const answerB = { x: 400, y: 630, w: 240, h: 80, label: "Age 3", value: 3 };  
  
const correctAnswer = 1;  
  
function drawQuiz() {
```

```

// draw image background (fallback if not loaded yet)
if (quizImg1) image(quizImg1, 0, 0, width, height);
else background(230);

// top bar for readability
rectMode(CORNER);
noStroke();
fill(255, 220);
rect(0, 0, width, 120);

// title
fill(30, 50, 60);
textAlign(CENTER, CENTER);
textSize(32);
text("How old am I here?", width / 2, 60);

// answer buttons (reuses drawButton from start.js)
drawButton(answerA);
drawButton(answerB);

// cursor feedback
cursor(isHover(answerA) || isHover(answerB) ? HAND : ARROW);
}

function quizMousePressed() {
  if (isHover(answerA)) checkQuizAnswer(answerA.value);
  else if (isHover(answerB)) checkQuizAnswer(answerB.value);
}

function checkQuizAnswer(value) {
  currentScreen = value === correctAnswer ? "win" : "lose";
}

```

Why this works with your setup

- `quizImg1` is loaded globally in `main.js`
- `drawButton()` exists globally from `start.js`
- `isHover()` exists globally from `main.js`
- `currentScreen` is global in `main.js`

All scripts share the same global scope because they're loaded by `index.html`.



Step 3 — Make sure `index.html` loads `quiz.js`

Add this **before** `main.js`:

```

<script src="quiz.js"></script>
<script src="main.js"></script>

```

(Keep `main.js` last.)

✓ Step 4 — Quick naming check (this is where people get stuck)

These screen names must match everywhere:

- Start screen sets: `currentScreen = "quiz"`
 - Router checks: `else if (currentScreen === "quiz") drawQuiz();`
 - Instructions is: `"instr"` (not `"instructions"`)
-

Next after this works

To add more rounds, we replace `answerA/answerB/correctAnswer` with an array of question objects and an index (`qIndex`). But don't expand until Round 1 shows the image + buttons and routes to win/lose properly.

If you paste your `index.html` bottom script section, I'll tell you the exact correct order for your files.

so now send me what exactly i should put in start and what i should put in quiz

Perfect.

Here is the **clean, minimal, correct split** for your project — matching the GBDA multi-file screen pattern you started with.

I am only showing:

-  what goes in **start.js**
-  what goes in **quiz.js**

(no extra theory, no refactors)

This assumes:

- `drawButton()` already exists in `start.js`
- `quizImg1` is loaded in `main.js`
- router already has `"quiz"`

✓ start.js (ONLY the start screen)

Put this in **start.js**:

```
function drawStart() {
  // Background image (fallback if not loaded)
  if (startBg) image(startBg, 0, 0, width, height);
  else background(180, 225, 220);

  // overlay for readability
  rectMode(CORNER);
  noStroke();
  fill(255, 255, 255, 140);
  rect(0, 0, width, height);

  // Title
  fill(30, 50, 60);
  textSize(46);
  textAlign(CENTER, CENTER);
  text("Meet Ksenia", width / 2, 180);

  const startBtn = { x: width / 2, y: 320, w: 240, h: 80, label: "START" };
  const instrBtn = { x: width / 2, y: 430, w: 240, h: 80, label:
"INSTRUCTIONS" };

  drawButton(startBtn);
  drawButton(instrBtn);

  cursor(isHover(startBtn) || isHover(instrBtn) ? HAND : ARROW);
}

function startMousePressed() {
  const startBtn = { x: width / 2, y: 320, w: 240, h: 80 };
  const instrBtn = { x: width / 2, y: 430, w: 240, h: 80 };

  if (isHover(startBtn)) currentScreen = "quiz";
  else if (isHover(instrBtn)) currentScreen = "instr";
}

function startKeyPressed() {
  if (keyCode === ENTER) currentScreen = "quiz";
  if (key === "i" || key === "I") currentScreen = "instr";
}

// shared button drawing helper
function drawButton({ x, y, w, h, label }) {
  rectMode(CENTER);
  const hover = isHover({ x, y, w, h });

  noStroke();
```

```

if (hover) {
  fill(255, 200, 150, 220);
  drawingContext.shadowBlur = 20;
  drawingContext.shadowColor = color(255, 180, 120);
} else {
  fill(255, 240, 210, 210);
  drawingContext.shadowBlur = 8;
  drawingContext.shadowColor = color(220, 220, 220);
}

rect(x, y, w, h, 14);

drawingContext.shadowBlur = 0;

fill(40, 60, 70);
textSize(28);
textAlign(CENTER, CENTER);
text(label, x, y);
}

```

That's it.

⚠ No quiz code in this file.

✓ quiz.js (ONLY the quiz screen)

Put this in **quiz.js**:

```

// -----
// Quiz screen (round 1)
// -----

const answerA = { x: 400, y: 520, w: 240, h: 80, label: "Age 1", value: 1 };
const answerB = { x: 400, y: 630, w: 240, h: 80, label: "Age 3", value: 3 };

const correctAnswer = 1;

function drawQuiz() {

  // show image
  if (quizImg1) {
    image(quizImg1, 0, 0, width, height);
  } else {
    background(230);
  }

  // top overlay
  rectMode(CORNER);
  noStroke();
  fill(255, 220);

```

```

    rect(0, 0, width, 120);

    fill(30, 50, 60);
    textAlign(CENTER, CENTER);
    textSize(32);
    text("How old am I here?", width / 2, 60);

    drawButton(answerA);
    drawButton(answerB);

    cursor(isHover(answerA) || isHover(answerB) ? HAND : ARROW);
}

function quizMousePressed() {
    if (isHover(answerA)) checkQuizAnswer(answerA.value);
    else if (isHover(answerB)) checkQuizAnswer(answerB.value);
}

function checkQuizAnswer(value) {
    if (value === correctAnswer) {
        currentScreen = "win";
    } else {
        currentScreen = "lose";
    }
}

```

That's all.

⚠ ONE VERY IMPORTANT LINE (you probably missed this)

In your **main.js**, your mouse router must include quiz:

```
else if (currentScreen === "quiz") quizMousePressed();
```

So your mousePressed router looks like:

```

function mousePressed() {
    if (currentScreen === "start") startMousePressed();
    else if (currentScreen === "instr") instrMousePressed();
    else if (currentScreen === "quiz") quizMousePressed();
    else if (currentScreen === "game") gameMousePressed?();
    else if (currentScreen === "win") winMousePressed?();
    else if (currentScreen === "lose") loseMousePressed?();
}

```

If this line is missing → buttons will draw but not work.

AND your draw router must already have:

```
else if (currentScreen === "quiz") drawQuiz();
```

AND in index.html (order matters)

Before `main.js`:

```
<script src="start.js"></script>
<script src="instructions.js"></script>
<script src="quiz.js"></script>
<script src="game.js"></script>
<script src="win.js"></script>
<script src="lose.js"></script>
<script src="main.js"></script>
```

After you paste exactly this:

You should have:

START → quiz screen

Image visible

Two buttons

Click → win / lose

Once this works, I can show you how to add **round 2 and round 3** properly (without copy-pasting more files).